

Visualization Literacy Analysis

Derek Harter, Shulan Lu

11/12/2025

Read in data files

Raw data is in the ../AccData subdirectory in two excel spreadsheet files per participant. The following code combines into single dataframe of all data.

```
#grab files from google drive (only have to do this once)
# source("getFromGoogleDrive.R")

#get the first 6? characters of each data file
#get unique values of these
# this is list of subject ids

raw_file_names <- list.files("../AccData")
first_six <- substr(raw_file_names, 1, 6)
sub_ids <- unique(first_six)

# fast_RT = data.frame(ParticipantId = character(),
#                        TrialName = character(),
#                        type = character(),
#                        time = numeric()
# )

all_data <- NULL

for (i in 1:length(sub_ids)){

  if (grepl("~$", sub_ids[i], fixed = T)){
    next
  }

  temp_file1 <- read_xlsx(paste0("../AccData/", sub_ids[i], "_1.xlsx")) %>%
    slice(1:17) %>%
    select(-starts_with("Order")) %>%
    rename(correct = 8)

  temp_file2 <- read_xlsx(paste0("../AccData/", sub_ids[i], "_2.xlsx")) %>%
    slice(1:17) %>%
    select(-starts_with("Order")) %>%
    rename(correct = 8)

  new_temp <- temp_file1 %>%
    bind_cols(temp_file2$correct) %>%
```

```

  rename(Correct_1 = correct, Correct_2 = "...9") %>%
  mutate(AnswerRT = TimeToBeginInput - TimeToReadQuestion)

all_data <- all_data %>%
  bind_rows(new_temp)
}

all_data <- all_data %>%
  rename(readRT = TimeToReadQuestion, totalRT = TimeToBeginInput)

#read in trialtype key (I created this from an early version of the previous paper)
trial_type_key <- read.csv("../trial_type_key.csv", stringsAsFactors = F)

all_data <- all_data %>%
  mutate(TrialType = trial_type_key$TrialType[match(TrialName, trial_type_key$TrialName)]) %>%
  mutate(TrialType = paste0("Type", TrialType))

# remove temporary dataframes to keep workspace cleaner as we go forward
rm(new_temp)
rm(temp_file1)
rm(temp_file2)
rm(trial_type_key)

```

Basic checks

Just a sanity check of all the data. Summary of number of subjects in each of the 3 condition types.

```

#how many participants per condition
all_data %>%
  group_by(ParticipantId, Condition) %>%
  summarize(ntrials = n()) %>%
  group_by(Condition) %>%
  summarize(nsubs = n()) %>%
  summary()

```

`summarise()` has grouped output by 'ParticipantId'. You can override using the `.groups` argument.

```

##   Condition      nsubs
## Length:3      Min.   :33.00
## Class :character 1st Qu.:36.00
## Mode  :character Median :39.00
##                Mean   :40.67
##                3rd Qu.:44.50
##                Max.   :50.00

```

#why is the balance so off?

Remove outliers based on RT

Removing outliers, resulting in data frame named `all_data_no_outliers`.

1. All trials where the **AnswerRT** is less than 2 seconds are removed first, which assumes these reactions were too fast so were done without an actual considered response.
2. All trials where the **AnswerRT** is 3 standard deviations different from the mean are removed next.

Table shows summary of mean and standard deviation of answer RT by each of the trial question types, as well as the 3 standard deviation upper and lower bound of the reaction time.

Displayed histogram is a sanity check of the number of trials each participant in all of the data have done. Most participants should do 16 trials. Less than that are from dropped data for outliers.

```
#removing on trial-by-trial basis

#remove answerRTs below 2000ms first
all_data_remove <- all_data %>%
  filter(AnswerRT >= 2000)
dim(all_data)[1]

## [1] 2074
dim(all_data_remove)[1]

## [1] 2067
#drops 7 trials

# also remove AnswerRT that are more than 3 standard deviations away from
# the mean
all_data_no_outliers <- all_data_remove %>%
  group_by(TrialName) %>%
  filter(!(abs(AnswerRT - mean(AnswerRT)) > 3.0*sd(AnswerRT)))
dim(all_data_no_outliers)[1]

## [1] 2020
#drops 47 more trials

rt_data_summary <- all_data_no_outliers %>%
  group_by(TrialName) %>%
  summarize(meanAnswerRT = mean(AnswerRT, na.rm = T),
            sdAnswerRT = sd(AnswerRT, na.rm = T),
            LB = meanAnswerRT - 3*sdAnswerRT,
            UB = meanAnswerRT + 3*sdAnswerRT)
summary(rt_data_summary)

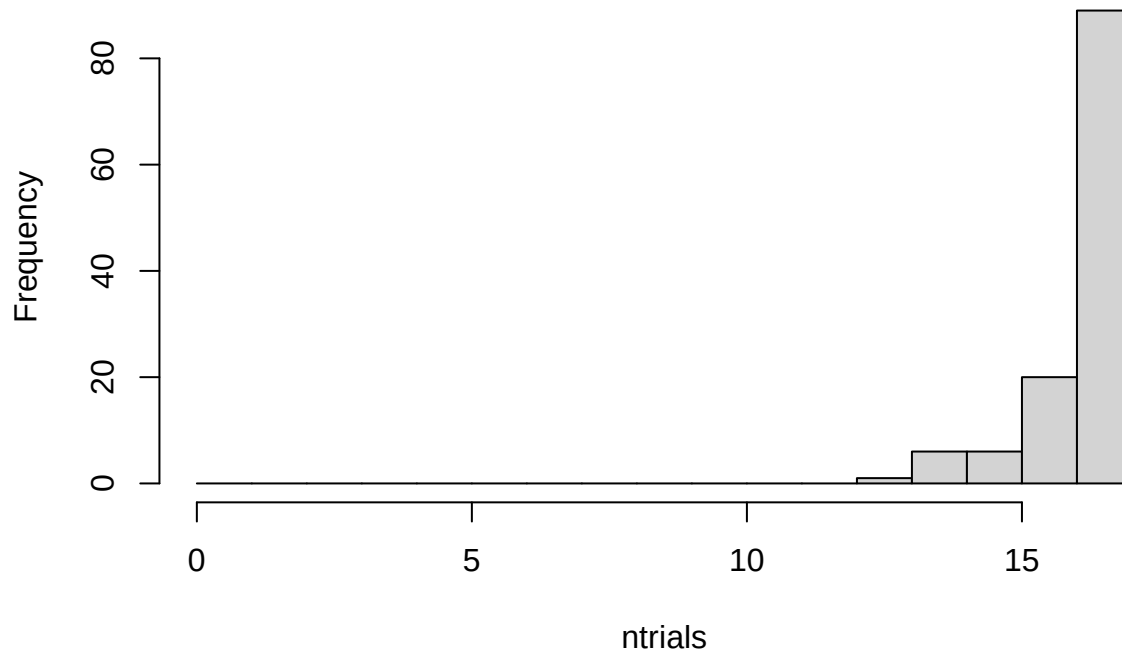
##   TrialName      meanAnswerRT    sdAnswerRT      LB      UB
## Length:17      Min.   : 8963      Min.   : 5156      Min.   :-67692      Min.   : 24429
## Class :character 1st Qu.:25275      1st Qu.:13263      1st Qu.: -35148      1st Qu.: 65715
## Mode  :character Median :29913      Median :19809      Median : -26012      Median : 88817
##              Mean  :33477      Mean  :20886      Mean  : -29182      Mean  : 96137
##              3rd Qu.:44519      3rd Qu.:26920      3rd Qu.: -17508      3rd Qu.:123649
##              Max.  :61772      Max.  :43154      Max.   : -6504      Max.   :191235

rm(rt_data_summary)

all_data_no_outliers %>%
  group_by(ParticipantId) %>%
```

```
summarize(ntrials = n()) %>%
with(hist(ntrials, breaks = 0:17))
```

Histogram of ntrials



```
rm(all_data_remove)
##maybe consider replacing outliers with means instead of removing them?
```

Extract Only One Type of Chart

Create a dataframe from the data with no outliers that contains only XChartQ1 through XChartQ4 trials for the following analysis.

We replace the `all_data_no_outliers` dataframe with one containing only the selected TrialName question type here, so past this point we are analyzing only this particular TrialName type.

We show the table again of the mean RT for this question type. The table should match the one done with all of the data, but we should only have the particular question trial name type now in the data.

We also again plot the histogram of the number of trials. Participants should have seen 4 of each particular question type, but after removing outliers some only have 3, 2 or 1 remaining.

```
all_data_no_outliers <- all_data_no_outliers %>%
  filter(grepl("BarChartQ[0-9]", TrialName))

rt_data_summary <- all_data_no_outliers %>%
  group_by(TrialName) %>%
  summarize(meanAnswerRT = mean(AnswerRT, na.rm = T),
            sdAnswerRT = sd(AnswerRT, na.rm = T),
            LB = meanAnswerRT - 3*sdAnswerRT,
```

```

UB = meanAnswerRT + 3*sdAnswerRT
summary(rt_data_summary)

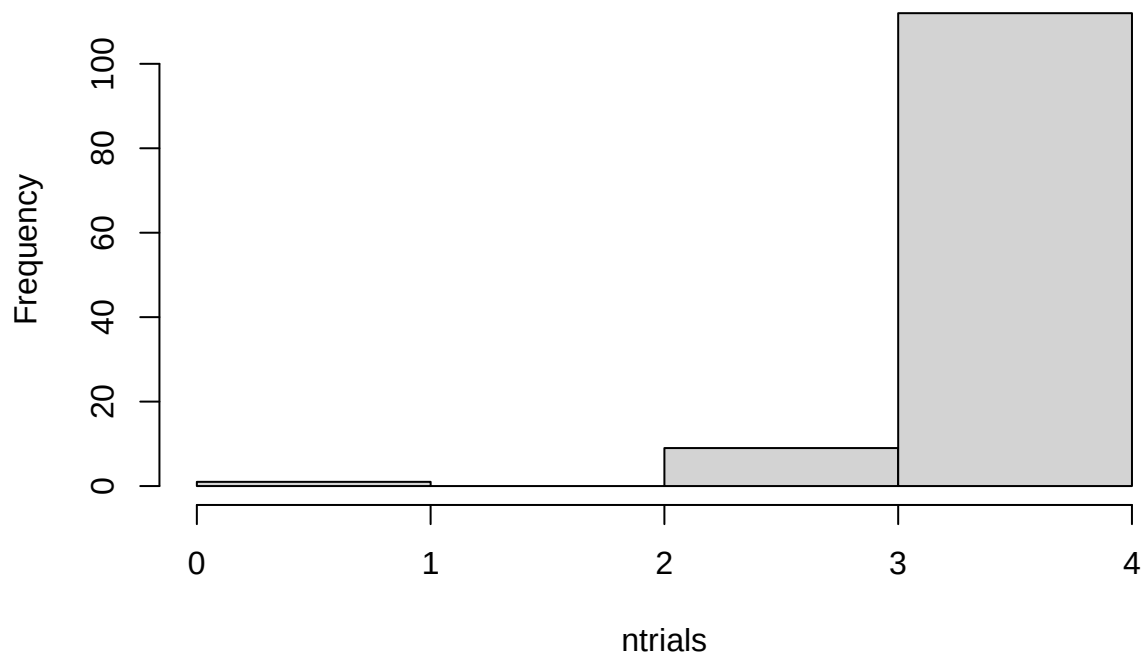
##   TrialName      meanAnswerRT      sdAnswerRT      LB      UB
## Length:4      Min.   : 8963    Min.   : 5156    Min.   : -17088  Min.   : 24429
## Class :character 1st Qu.:13267  1st Qu.: 7458    1st Qu.: -13507  1st Qu.: 35641
## Mode  :character Median :15195    Median : 9575    Median : -11144  Median : 43921
##              Mean   :16513    Mean   : 9328    Mean   : -11470  Mean   : 44497
##              3rd Qu.:18441    3rd Qu.:11445    3rd Qu.: -9107   3rd Qu.: 52777
##              Max.   :26701    Max.   :13005    Max.   : -6504   Max.   : 65715

rm(rt_data_summary)

all_data_no_outliers %>%
  group_by(ParticipantId) %>%
  summarize(ntrials = n()) %>%
  with(hist(ntrials, breaks = 0:4))

```

Histogram of ntrials



```

#compare answer times

trial_type_means <- all_data_no_outliers %>%
  group_by(ParticipantId, Condition, TrialType) %>%
  summarize(mean_answerRT = mean(AnswerRT), n = n())

## `summarise()` has grouped output by 'ParticipantId', 'Condition'. You can override using
## the `.groups` argument.

```

```

# first, make .csv files in wide format to double check in statview
answer_rt <- trial_type_means %>%
  select(ParticipantId, Condition, TrialType, mean_answerRT) %>%
  pivot_wider(names_from = TrialType, values_from = mean_answerRT)

# fill in na values with mean for type1, type2 and type3 RT
# the following is supposed to work, but we get NaN from calculation of mean here, not sure why...
#read_rt_type_wider_clean <- read_rt_type_wider %>% mutate(across(Type1, ~replace_na(., mean(., na.rm=T))))
answer_rt <- answer_rt %>% replace_na(list(Type1 = mean(answer_rt$Type1, na.rm=TRUE)))
answer_rt <- answer_rt %>% replace_na(list(Type2 = mean(answer_rt$Type2, na.rm=TRUE)))
answer_rt <- answer_rt %>% replace_na(list(Type3 = mean(answer_rt$Type3, na.rm=TRUE)))

answer_rt_summary <- answer_rt %>%
  group_by(Condition) %>%
  summarize(meanType1 = mean(Type1, na.rm = T),
            meanType2 = mean(Type2, na.rm = T),
            meanType3 = mean(Type3, na.rm=T))
kable(answer_rt_summary)

```

Condition	meanType1	meanType2	meanType3
VR	22614.11	15243.81	11429.11
VR Monitor	29303.13	16692.65	13590.41
VR Monitor Stereo	29819.09	15174.25	11997.74

```

rm(answer_rt_summary)

#rm(trial_type_means)

```

Alternative Analysis using glmm TMB package

The following analysis is for the heterogenous group condition analysis. See .

The following determines if a more complex model is justified.

```

model_glmm_hom = glmmTMB(mean_answerRT ~ Condition * TrialType + (1 | ParticipantId),
  data=trial_type_means, family = Gamma(link="log"),
  dispformula = ~ 1, REML = FALSE)
model_glmm_hetero = glmmTMB(mean_answerRT ~ Condition * TrialType + (1 | ParticipantId),
  data=trial_type_means, family = Gamma(link="log"),
  dispformula = ~ Condition, REML = FALSE)

anova(model_glmm_hom, model_glmm_hetero)

## Data: trial_type_means
## Models:
## model_glmm_hom: mean_answerRT ~ Condition * TrialType + (1 | ParticipantId), zi=~0, disp=~1
## model_glmm_hetero: mean_answerRT ~ Condition * TrialType + (1 | ParticipantId), zi=~0, disp=~Condition
##           Df    AIC    BIC logLik deviance Chisq Chi Df Pr(>Chisq)
## model_glmm_hom    11 7517.9 7560.6 -3747.9   7495.9
## model_glmm_hetero  13 7521.4 7571.9 -3747.7   7495.4 0.4736     2    0.7892

```

Try overall effect reporting

```
fixed_effects_anova <- Anova(model_glmm_hom, type="III")
kable(fixed_effects_anova)
```

	Chisq	Df	Pr(>Chisq)
(Intercept)	16539.823090	1	0.0000000
Condition	6.164242	2	0.0458619
TrialType	61.158375	2	0.0000000
Condition:TrialType	4.027610	4	0.4022821

```
#summary(fixed_effects_anova, level=0.95)
```

EMM on Condition and Interaction

If the glmm analysis is not significant, just use the homogeneous model. Otherwise report on the more complex heterogeneous model?

```
emm_main_condition <- emmeans(model_glmm_hom,
                               specs = ~ Condition,
                               type = "response")
```

NOTE: Results may be misleading due to involvement in interactions

```
condition_results <- pairs(emm_main_condition)
```

```
summary(condition_results, level=0.95)
```

```
## contrast          ratio      SE df null z.ratio p.value
## VR / VR Monitor    0.857 0.0740 Inf   1 -1.782 0.1755
## VR / VR Monitor Stereo 0.902 0.0817 Inf   1 -1.137 0.4909
## VR Monitor / VR Monitor Stereo 1.052 0.1000 Inf   1  0.532 0.8555
##
## Results are averaged over the levels of: TrialType
## P value adjustment: tukey method for comparing a family of 3 estimates
## Tests are performed on the log scale
```

```
confint(condition_results, level=0.95)
```

```
## contrast          ratio      SE df asymp.LCL asymp.UCL
## VR / VR Monitor    0.857 0.0740 Inf   0.700    1.05
## VR / VR Monitor Stereo 0.902 0.0817 Inf   0.729    1.12
## VR Monitor / VR Monitor Stereo 1.052 0.1000 Inf   0.841    1.32
##
## Results are averaged over the levels of: TrialType
## Confidence level used: 0.95
## Conf-level adjustment: tukey method for comparing a family of 3 estimates
## Intervals are back-transformed from the log scale
```

```
#kable(condition_results)
```

```
emm_interaction <- emmeans(model_glmm_hom,
                            specs = ~ Condition * TrialType,
                            type = "response")
```

```
simple_effects_results <- pairs(emm_interaction, by = "TrialType")

summary(simple_effects_results, level=0.95)

## TrialType = Type1:
## contrast ratio SE df null z.ratio p.value
## VR / VR Monitor 0.789 0.0921 Inf 1 -2.026 0.1061
## VR / VR Monitor Stereo 0.768 0.0938 Inf 1 -2.161 0.0780
## VR Monitor / VR Monitor Stereo 0.973 0.1250 Inf 1 -0.216 0.9746
##
## TrialType = Type2:
## contrast ratio SE df null z.ratio p.value
## VR / VR Monitor 0.917 0.1080 Inf 1 -0.741 0.7390
## VR / VR Monitor Stereo 0.984 0.1230 Inf 1 -0.133 0.9902
## VR Monitor / VR Monitor Stereo 1.073 0.1390 Inf 1 0.542 0.8508
##
## TrialType = Type3:
## contrast ratio SE df null z.ratio p.value
## VR / VR Monitor 0.871 0.1010 Inf 1 -1.189 0.4600
## VR / VR Monitor Stereo 0.972 0.1180 Inf 1 -0.235 0.9700
## VR Monitor / VR Monitor Stereo 1.116 0.1430 Inf 1 0.853 0.6700
##
## P value adjustment: tukey method for comparing a family of 3 estimates
## Tests are performed on the log scale
```

```
confint(simple_effects_results, level=0.95)

## TrialType = Type1:
## contrast ratio SE df asymp.LCL asymp.UCL
## VR / VR Monitor 0.789 0.0921 Inf 0.601 1.04
## VR / VR Monitor Stereo 0.768 0.0938 Inf 0.577 1.02
## VR Monitor / VR Monitor Stereo 0.973 0.1250 Inf 0.720 1.31
##
## TrialType = Type2:
## contrast ratio SE df asymp.LCL asymp.UCL
## VR / VR Monitor 0.917 0.1080 Inf 0.696 1.21
## VR / VR Monitor Stereo 0.984 0.1230 Inf 0.734 1.32
## VR Monitor / VR Monitor Stereo 1.073 0.1390 Inf 0.791 1.45
##
## TrialType = Type3:
## contrast ratio SE df asymp.LCL asymp.UCL
## VR / VR Monitor 0.871 0.1010 Inf 0.664 1.14
## VR / VR Monitor Stereo 0.972 0.1180 Inf 0.731 1.29
## VR Monitor / VR Monitor Stereo 1.116 0.1430 Inf 0.826 1.51
##
## Confidence level used: 0.95
## Conf-level adjustment: tukey method for comparing a family of 3 estimates
## Intervals are back-transformed from the log scale

#kable(simple_effects_results)
```


EMM

```
emm_main_condition <- emmeans(model_glm_hom,
                              specs = ~ Condition,
                              type = "response")

## NOTE: Results may be misleading due to involvement in interactions
summary(emm_main_condition, level=0.95)

## Condition      response    SE  df asymp.LCL asymp.UCL
## VR             15184   871 Inf    13568    16992
## VR Monitor     17708  1140 Inf    15602    20098
## VR Monitor Stereo 16832 1180 Inf    14667    19316
##
## Results are averaged over the levels of: TrialType
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
confint(emm_main_condition, level=0.95)

## Condition      response    SE  df asymp.LCL asymp.UCL
## VR             15184   871 Inf    13568    16992
## VR Monitor     17708  1140 Inf    15602    20098
## VR Monitor Stereo 16832 1180 Inf    14667    19316
##
## Results are averaged over the levels of: TrialType
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
contrast_coefficients <- c(-1, 0.5, 0.5)

# Create a named list for the contrast matrix
contrast_matrix <- list(
  VR_vs_Monitors_Collapsed = contrast_coefficients
)

comparison_results <- contrast(emm_main_condition,
                              method = contrast_matrix)
summary(comparison_results, level = 0.95)

## contrast          ratio    SE  df null z.ratio p.value
## VR_vs_Monitors_Collapsed 1.14 0.0848 Inf    1  1.723  0.0849
##
## Results are averaged over the levels of: TrialType
## Tests are performed on the log scale
confint(comparison_results, level=0.95)

## contrast          ratio    SE  df asymp.LCL asymp.UCL
## VR_vs_Monitors_Collapsed 1.14 0.0848 Inf    0.982    1.32
##
## Results are averaged over the levels of: TrialType
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
Try explicitly naming the coefficients to remove all ambiguity
```

```

comparison_results <- contrast(
  emm_main_condition,
  method = list(
    VR_vs_Monitors_Collapsed = c('VR' = -1, 'VR Monitor' = 0.5, 'VR Monitor Stereo' = 0.5)
  )
)

summary(comparison_results, infer = TRUE)

## contrast          ratio      SE df asymp.LCL asymp.UCL null z.ratio p.value
## VR_vs_Monitors_Collapsed  1.14 0.0848 Inf      0.982      1.32    1   1.723  0.0849
##
## Results are averaged over the levels of: TrialType
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
## Tests are performed on the log scale

confint(comparison_results, level = 0.95)

## contrast          ratio      SE df asymp.LCL asymp.UCL
## VR_vs_Monitors_Collapsed  1.14 0.0848 Inf      0.982      1.32
##
## Results are averaged over the levels of: TrialType
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale

contrast_coefficients <- c(-0.5, 1, -0.5)

# Create a named list for the contrast matrix
contrast_matrix <- list(
  VRMonitor_vs_Others_Collapsed = contrast_coefficients
)

comparison_results <- contrast(emm_main_condition,
  method = contrast_matrix)
summary(comparison_results, level = 0.95)

## contrast          ratio      SE df null z.ratio p.value
## VRMonitor_vs_Others_Collapsed  1.11 0.0873 Inf    1   1.297  0.1945
##
## Results are averaged over the levels of: TrialType
## Tests are performed on the log scale

confint(comparison_results, level=0.95)

## contrast          ratio      SE df asymp.LCL asymp.UCL
## VRMonitor_vs_Others_Collapsed  1.11 0.0873 Inf      0.949      1.29
##
## Results are averaged over the levels of: TrialType
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale

Try explicitly naming the coefficients to remove all ambuugity

comparison_results <- contrast(
  emm_main_condition,
  method = list(

```

```
VRMonitor_vs_others_collapsed = c('VR' = -0.5, 'VR Monitor' = 1, 'VR Monitor Stereo' = -0.5)
)
summary(comparison_results, infer = TRUE)
```

```
## contrast ratio SE df asymp.LCL asymp.UCL null z.ratio p.value
## VRMonitor_vs_others_collapsed 1.11 0.0873 Inf 0.949 1.29 1 1.297 0.1945
##
## Results are averaged over the levels of: TrialType
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
## Tests are performed on the log scale
```

```
confint(comparison_results, level = 0.95)
```

```
## contrast ratio SE df asymp.LCL asymp.UCL
## VRMonitor_vs_others_collapsed 1.11 0.0873 Inf 0.949 1.29
##
## Results are averaged over the levels of: TrialType
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
```